

Echo 多端同步架构设计文档

项目代号: Echo

文档版本: v1.0

发布日期: 2026年2月27日

架构师: 莫循瑾木

适用范围: 微信小程序原生开发 + Flutter App 独立开发

1. 架构设计目标

1.1 核心目标

本架构设计旨在解决**微信小程序（原生开发）与Flutter App（iOS/Android）**分开开发场景下的多端功能同步、数据一致性和开发协同问题[1][2]。

1.2 设计原则

- **大中台，小前台**: 业务逻辑、状态管理、数据计算全部下沉至后端，前端仅负责展示与交互[3]。
 - **API First**: 所有新功能开发前，后端必须先输出接口文档，前端并行开发[4]。
 - **端无关设计**: 数据库设计与业务实体完全独立于"端"，不允许出现"小程序专用字段"或"App专用接口"[5]。
 - **统一鉴权**: 小程序和App共用同一套用户身份体系（基于UnionID + JWT）[6]。
-

2. 整体架构图

Figure 1: Echo 多端统一后端架构示意图

2.1 三层架构

前端层 (Multi-Frontend Layer)

- 微信小程序：使用原生 WXML/WXSS/JavaScript 开发[7]。
- Flutter App：使用 Dart + Flutter SDK 开发，编译为 iOS 和 Android 原生应用[8]。

网关与API层 (Unified Gateway & API Layer)

- 统一 RESTful API 网关（如 /api/v1/courses、/api/v1/user/progress）[9]。
- 统一 WebSocket 长连接（用于实时语音识别流式通信，地址如 wss://api.echo.com/asr）[10]。
- 端标识机制：所有请求必须携带 HTTP Header X-Client-Type（值为 weapp / flutter_ios / flutter_android）和 X-Client-Version（如 1.0.0）[11]。

后端服务层 (Backend Services Layer)

- 用户服务 (User Service)：账号注册、登录、UnionID绑定、JWT 生成与校验[12]。
- 课程服务 (Course Service)：课程列表、课程详情、句子获取、音频URL返回[13]。
- 训练服务 (Training Service)：学习进度记录、ASR结果比对、评分计算、弱项收集[14]。
- AI服务 (AI Service)：调用大语言模型API生成语境解析、生词释义、发音技巧提示[15]。
- 支付服务 (Payment Service)：统一处理微信支付、支付宝、Apple IAP内购订单[16]。

数据存储层 (Data Storage Layer)

- MySQL：用户信息、课程元数据、学习记录、订单数据[17]。
 - Redis：会话缓存、热点数据缓存、分布式锁[18]。
 - 对象存储 (OSS/COS)：音频文件、封面图片[19]。
-

3. 统一身份认证与账号同步

3.1 用户主键设计

核心原则：绝不使用微信OpenID作为用户主键[20]

- 数据库用户表（users）的主键为 user_id（UUID或自增ID）。
- 微信生态相关字段单独存储：wechat_openid（小程序OpenID）、wechat_unionid（开放平台UnionID）、wechat_app_openid（App的OpenID）。

3.2 UnionID统一账号机制

前提条件：小程序和App必须绑定到同一个微信开放平台账号下[21]。

登录流程（小程序端）

1. 用户在小程序点击"微信一键登录"，前端调用 wx.login() 获取临时 code。
2. 前端将 code 发送至后端接口 /api/v1/auth/weapp/login。
3. 后端调用微信接口 https://api.weixin.qq.com/sns/jscode2session，用 code 换取 openid 和 unionid。
4. 后端根据 unionid 在数据库中查找用户：
 - 如果 unionid 已存在，返回该用户的 user_id 和 JWT Token。
 - 如果不存在，创建新用户记录，生成 user_id，存储 unionid 和 openid，返回 JWT Token。
5. 前端将 JWT Token 存储在本地（小程序使用 wx.setStorageSync），后续所有请求在 Header 中携带 Authorization: Bearer <token>。

登录流程（Flutter App端）

1. 用户在App点击"微信登录"，调用微信开放SDK获取授权 code。
2. 前端将 code 发送至后端接口 /api/v1/auth/app/wechat_login。
3. 后端调用微信接口获取 openid 和 unionid。
4. 后端根据 unionid 查找用户（逻辑同小程序），如果已存在则直接返回JWT，用户无需重新注册[22]。

手机号绑定机制

- 用户在小程序或App中绑定手机号时，后端将手机号写入 users 表的 phone 字段。
- 后续用户可以使用"手机号+验证码"登录，后端根据手机号查找 user_id 并返回JWT[23]。

3.3 JWT Token设计

- **过期时间：**设置为30天，前端在Token过期前7天自动静默刷新。
- **安全性：**使用RS256算法签名，后端持有私钥，前端仅验证公钥。

4. 统一API接口设计规范

4.1 RESTful API规范

URL结构

常用资源示例

接口路径	方法	说明
/api/v1/courses	GET	获取课程列表（支持分页、筛选）
/api/v1/courses/{id}	GET	获取课程详情
/api/v1/courses/{id}/sentences	GET	获取课程的所有句子
/api/v1/user/progress	GET	获取用户学习进度
/api/v1/user/progress	POST	提交单句训练结果
/api/v1/user/weakness	GET	获取弱项句库
/api/v1/ai/analyze	POST	请求AI语境解析
/api/v1/payment/create	POST	创建支付订单

Table 1: 核心API接口列表

请求Header规范

响应格式统一

成功响应：

错误响应：

4.2 WebSocket协议（实时语音识别）

连接地址

消息格式（前端→后端）

消息格式（后端→前端）

5. 数据库设计（端无关原则）

5.1 用户表（users）

字段名	类型	说明
user_id	VARCHAR(64)	主键，全局唯一用户ID
wechat_unionid	VARCHAR(128)	微信UnionID（可空）
wechat_openid_weapp	VARCHAR(128)	小程序OpenID（可空）
wechat_openid_app	VARCHAR(128)	App的OpenID（可空）
phone	VARCHAR(20)	手机号（可空）
nickname	VARCHAR(50)	昵称
avatar_url	VARCHAR(255)	头像URL
vip_expire_at	DATETIME	VIP到期时间 （NULL表示未开通）
created_at	DATETIME	注册时间
updated_at	DATETIME	最后更新时间

Table 2: 用户表结构

5.2 学习进度表（user_study_logs）

字段名	类型	说明
log_id	BIGINT	主键，自增ID
user_id	VARCHAR(64)	外键，关联用户
course_id	VARCHAR(64)	外键，关联课程
sentence_id	VARCHAR(64)	外键，关联句子
recognized_text	TEXT	ASR识别的文本
accuracy_score	INT	识别准确度（0-100）
is_weak	BOOLEAN	是否标记为弱项（准确度<60%）
client_type	VARCHAR(20)	记录端类型 (weapp/flutter_ios/flutter_androi
created_at	DATETIME	训练时间

Table 3: 学习进度表结构

关键设计说明

- **最小同步单元**：进度同步细化到"句"级别，而不是"课程"级别[24]。
- **端标识字段**：client_type 字段记录本次训练来自哪个端，用于后续数据分析，但不影响业务逻辑。
- **离线同步策略**：Flutter App支持离线训练，训练记录先存本地SQLite，网络恢复后批量上传至后端。后端根据 created_at 时间戳去重合并[25]。

5.3 课程与句子表 (courses & sentences)

课程表 (courses)

字段名	类型	说明
course_id	VARCHAR(64)	主键
title	VARCHAR(100)	课程标题
description	TEXT	课程简介
category	VARCHAR(20)	分类（toefl/ielts/cet4/cet6/daily）
difficulty	VARCHAR(20)	难度 (beginner/intermediate/advance)
cover_url	VARCHAR(255)	封面图片URL
total_sentences	INT	总句数
is_vip_only	BOOLEAN	是否仅VIP可访问
created_at	DATETIME	创建时间

Table 4: 课程表结构

句子表 (**sentences**)

字段名	类型	说明
sentence_id	VARCHAR(64)	主键
course_id	VARCHAR(64)	外键，关联课程
sentence_index	INT	句子在课程中的顺序（从1开始）
audio_url	VARCHAR(255)	音频文件URL（存储在OSS/COS）
text_en	TEXT	英文原文
text_cn	TEXT	中文翻译
start_time	FLOAT	音频起始时间（秒）
end_time	FLOAT	音频结束时间（秒）

Table 5: 句子表结构

6. 业务逻辑下沉策略

6.1 核心原则：前端只做展示，后端做判断

错误示范

前端（小程序或Flutter）代码中写：

正确做法

前端仅发送识别结果给后端：
后端返回判断结果：
前端根据 is_pass 字段决定是否显示成功动画[26]。

6.2 计算逻辑后端化清单

- **识别准确度计算**：后端对比ASR识别文本与原文，计算编辑距离，返回准确度分数。
- **弱项判定**：后端根据阈值（如准确度<60%）判断是否加入弱项库。
- **VIP权限校验**：后端根据JWT中的 vip_expire_at 判断用户是否有权访问VIP课程或AI解析。
- **学习数据统计**：累计训练时长、完成句数等统计在后端聚合计算，前端直接请求 /api/v1/user/stats 获取结果。

7. 团队开发协同规范

7.1 API First开发流程

1. **需求评审**：产品经理提出新功能需求（如"新增连读弱读AI评分"）。
2. **接口设计**：后端负责人在Swagger/Apifox中设计接口文档（包括请求参数、响应格式、错误码）。
3. **接口评审**：前端（小程序、Flutter）、后端、测试共同评审接口设计，确认无误。
4. **Mock数据**：后端提供Mock Server，前端基于Mock数据并行开发。
5. **联调上线**：后端完成接口开发后，前后端联调测试，通过后发布上线[27]。

7.2 版本控制与灰度发布

功能开关表 (feature_flags)

功能名称	小程序开关	App开关	描述
ai_tutor	true	false	AI语境解析功能
voice_score_v2	false	false	新版发音评分算法
social_ranking	true	true	学习排行榜

Table 6: 功能开关配置表
灰度策略

- 后端接口 `/api/v1/feature_flags` 返回当前用户可用的功能列表。
- 前端根据返回的开关决定是否显示某个功能入口。
- 如果App审核比小程序慢，可以先开启小程序的功能开关，等App过审后再统一开启[28]。

7.3 代码仓库管理

- **仓库划分：**
 - `echo-weapp`：微信小程序前端代码。
 - `echo-flutter`：Flutter App前端代码。
 - `echo-backend`：后端API服务代码。
 - `echo-admin`：管理后台代码。
 - **分支策略：**采用Git Flow模型，`main` 分支为生产环境，`develop` 分支为开发环境，功能开发在 `feature/*` 分支上进行。
 - **版本号同步：**小程序、App、后端的版本号独立管理，但在发版时统一记录在 `CHANGELOG.md` 中[29]。
-

8. 性能优化与容灾设计

8.1 音频加载优化

- **CDN加速：**所有音频文件上传至阿里云OSS或腾讯云COS，并开启CDN加速，确保全国各地用户的加载速度[30]。
- **预加载策略：**用户播放第N句时，后台预加载第N+1句和第N+2句的音频，减少等待时间。
- **音频压缩：**使用MP3格式，码率设置为64kbps（清晰度足够），单句音频文件控制在50KB以内。

8.2 ASR服务容灾

- **多云备份：**主服务使用腾讯云ASR，备用服务接入阿里云ASR，当主服务响应超时或错误率过高时自动切换[31]。
- **降级策略：**如果ASR服务完全不可用，前端提示用户"语音识别服务暂时不可用，请稍后再试"，但允许用户继续听音频和查看原文。

8.3 数据库读写分离

- **主从架构**：主库负责写操作（用户进度提交、订单创建），从库负责读操作（课程列表、学习数据统计）。
 - **缓存策略**：热点数据（如课程列表、用户VIP状态）使用Redis缓存，TTL设置为5分钟[32]。
-

9. 安全与合规

9.1 数据加密

- **传输加密**：所有API请求必须使用HTTPS，WebSocket使用WSS。
- **敏感数据加密**：用户手机号、微信UnionID在数据库中使用AES-256加密存储。
- **Token刷新机制**：JWT Token过期后，前端调用 `/api/v1/auth/refresh` 接口刷新Token，避免用户频繁重新登录[33]。

9.2 隐私合规

- **用户协议与隐私政策**：小程序和App首次启动时，必须弹窗让用户同意《用户协议》和《隐私政策》。
 - **数据最小化原则**：仅收集必要的用户信息（昵称、头像、手机号），不过度索取权限。
 - **数据删除权**：用户可在个人中心申请注销账号，后端在30天内删除所有相关数据[34]。
-

10. 监控与运维

10.1 核心监控指标

指标类型	指标名称	告警阈值
性能	API平均响应时间	>500ms
性能	WebSocket连接成功率	<95%
业务	ASR识别失败率	>5%
业务	支付成功率	<98%
系统	服务器CPU使用率	>80%
系统	数据库连接池满载	>90%

Table 7: 核心监控指标与告警阈值

10.2 日志收集

- 使用ELK（Elasticsearch + Logstash + Kibana）或阿里云SLS收集后端日志。
- 前端（小程序、Flutter）上报关键操作日志（如登录、训练开始、支付等）至后端统一日志系统。
- 日志保留期：30天[35]。

11. 总结与实施路线

11.1 架构优势

- **统一后端**：小程序和App共享同一套API，减少50%的后端开发量[36]。
- **数据一致**：用户在小程序和App之间无缝切换，学习进度实时同步。
- **灵活扩展**：未来接入Web端或其他平台（如鸿蒙），只需开发新的前端，后端无需修改。

11.2 实施步骤

Phase 1：基础设施搭建（第1-2周）

- 搭建后端API服务框架（Node.js + MySQL + Redis）。
- 设计并创建数据库表结构（用户表、课程表、进度表）。
- 实现统一身份认证接口（微信登录、JWT生成）。

Phase 2：核心功能开发（第3-6周）

- 后端实现课程管理、学习进度记录、ASR调用与文本对齐接口。

- 小程序前端实现课程大厅、核心训练器、个人中心。
- Flutter前端并行开发（基于Mock数据）。

Phase 3: 联调与测试（第7-8周）

- 小程序与后端联调，发布Beta版本。
- Flutter与后端联调，发布TestFlight版本（iOS）和内测版（Android）。
- 测试用户跨端数据同步是否正常。

Phase 4: 上线与迭代（第9周起）

- 小程序提交微信审核并上线。
- Flutter App提交Apple App Store和各大Android应用商店审核并上线。
- 根据用户反馈持续迭代优化。

12. 参考资料

- [1] 腾讯云开发者社区. (2020). 打通小程序和移动应用APP，实现一云多端. <https://cloud.tencent.com/developer/article/1648756>
- [2] 稀土掘金. (2023). 利用Flutter和小程序容器技术打造多端适配的小程序解决方案. <https://juejin.cn/post/7217801491190349885>
- [3] 稀土掘金. (2023). 写给开发者的软件架构实战：前后端分离的架构设计. <https://juejin.cn/post/7309156772565073929>
- [4] 阿里云开发者社区. (2017). 微信小程序的Web API接口设计及常见接口实现. <https://developer.aliyun.com/article/325237>
- [5] CSDN博客. (2025). Litemall多端适配方案：PC、移动、小程序统一架构. https://blog.csdn.net/gitblog_00090/article/details/154893958
- [6] 腾讯云开发者社区. (2020). 打通小程序和移动应用APP，实现一云多端. <https://cloud.tencent.com/developer/article/1648756>
- [7] CSDN博客. (2025). 2025-06-03-微信小程序开发框架详解. <https://blog.csdn.net/tkhhhhh/article/details/148513071>
- [8] 江苏猪八戒网. (2025). 微信小程序开发与Flutter的技术差异. <https://js.zx.zbj.com/baike/4187.html>
- [9] 阿里云开发者社区. (2017). 微信小程序的Web API接口设计及常见接口实现. <https://developer.aliyun.com/article/325237>
- [10] 百度智能云. (2025). uniapp跨端语音处理全攻略：录音、识别与可视化实践. <https://cloud.baidu.com/article/3665151>

- [11] CSDN博客. (2024). 小程序开发与APP开发的区别. <https://blog.csdn.net/qiniuchangxiang/article/details/139522476>
- [12] 腾讯云开发者社区. (2020). 打通小程序和移动应用APP, 实现一云多端. <https://cloud.tencent.com/developer/article/1648756>
- [13] 阿里云开发者社区. (2017). 微信小程序的Web API接口设计及常见接口实现. <https://developer.aliyun.com/article/325237>
- [14] 稀土掘金. (2023). 写给开发者的软件架构实战: 前后端分离的架构设计. <https://juejin.cn/post/7309156772565073929>
- [15] 百度智能云. (2025). 微信小程序语音识别: 从原理到实践的全链路解析. <https://cloud.baidu.com/article/4052841>
- [16] 腾讯云. (2025). APP的开发方式及特点. <https://cloud.tencent.com/developer/article/2506730>
- [17] 稀土掘金. (2023). 写给开发者的软件架构实战: 前后端分离的架构设计. <https://juejin.cn/post/7309156772565073929>
- [18] 稀土掘金. (2023). 写给开发者的软件架构实战: 前后端分离的架构设计. <https://juejin.cn/post/7309156772565073929>
- [19] 腾讯云开发者社区. (2020). 打通小程序和移动应用APP, 实现一云多端. <https://cloud.tencent.com/developer/article/1648756>
- [20] 腾讯云开发者社区. (2020). 打通小程序和移动应用APP, 实现一云多端. <https://cloud.tencent.com/developer/article/1648756>
- [21] 腾讯云开发者社区. (2020). 打通小程序和移动应用APP, 实现一云多端. <https://cloud.tencent.com/developer/article/1648756>
- [22] 腾讯云开发者社区. (2020). 打通小程序和移动应用APP, 实现一云多端. <https://cloud.tencent.com/developer/article/1648756>
- [23] 阿里云开发者社区. (2017). 微信小程序的Web API接口设计及常见接口实现. <https://developer.aliyun.com/article/325237>
- [24] 稀土掘金. (2023). 写给开发者的软件架构实战: 前后端分离的架构设计. <https://juejin.cn/post/7309156772565073929>
- [25] 稀土掘金. (2023). 写给开发者的软件架构实战: 前后端分离的架构设计. <https://juejin.cn/post/7309156772565073929>
- [26] 稀土掘金. (2023). 写给开发者的软件架构实战: 前后端分离的架构设计. <https://juejin.cn/post/7309156772565073929>
- [27] 阿里云开发者社区. (2017). 微信小程序的Web API接口设计及常见接口实现. <https://developer.aliyun.com/article/325237>
- [28] 美团技术团队. (2020). 美团外卖Flutter动态化实践. <https://tech.meituan.com/2020/06/23/meituan-flutter-flap.html>

- [29] 博客园. (2025). 跨平台开发地图：2025跨平台技术简单总结. <https://www.cnblogs.com/developer-laoliu/articles/19513187>
- [30] 百度智能云. (2025). uniapp跨端语音处理全攻略：录音、识别与可视化实践. <https://cloud.baidu.com/article/3665151>
- [31] 百度智能云. (2025). 微信小程序语音识别：从原理到实践的全链路解析. <https://cloud.baidu.com/article/4052841>
- [32] 稀土掘金. (2023). 写给开发者的软件架构实战：前后端分离的架构设计. <https://juejin.cn/post/7309156772565073929>
- [33] 阿里云开发者社区. (2017). 微信小程序的Web API接口设计及常见接口实现. <https://developer.aliyun.com/article/325237>
- [34] CSDN博客. (2024). 小程序开发与APP开发的区别. <https://blog.csdn.net/qiniuchangxiang/article/details/139522476>
- [35] 稀土掘金. (2023). 写给开发者的软件架构实战：前后端分离的架构设计. <https://juejin.cn/post/7309156772565073929>
- [36] 腾讯云开发者社区. (2020). 打通小程序和移动应用APP，实现一云多端. <https://cloud.tencent.com/developer/article/1648756>
-

文档结束